

Go Gulf Experts — 4-Week Implementation Plan

AI-Powered Resume Screening & Job-Matching Platform for Gulf Region Careers

What it does: Job seekers upload a resume and select/paste a target Gulf-region job posting (UAE, Saudi Arabia, Qatar, Kuwait, Bahrain, Oman). The system parses the resume, extracts skills/experience, and computes an AI match score against the job description — showing matched skills, missing skills, and a history of past matches. Positions Go Gulf Experts as a smart pre-screening tool for candidates and recruiters targeting Gulf employers.

Stack: - Frontend: React + Tailwind (branded as Go Gulf Experts — logo, color scheme, landing page) - Backend: FastAPI (Python — keeps ML and API in one language) - ML/NLP: spaCy (parsing/NER) + Sentence-Transformers (embeddings) + cosine similarity - File parsing: `pdfplumber` / `python-docx` - Database: PostgreSQL - Deployment: Docker + Railway/Render (backend), Vercel (frontend)

Week 1 — Planning, Data, and Foundation

Goal: Spec locked, resume/job dataset ready, skeleton apps running, DB schema in place.

• Day 1 — Scoping & Branding

- Define MVP: upload resume → paste/select Gulf job posting → get match score + skill gaps + save to history
- Define brand basics for Go Gulf Experts: logo placeholder, color palette, tagline (e.g., "Your AI Guide to Gulf Careers")
- Out of scope for v1: bulk resume ranking, multi-language (Arabic) support, direct ATS/recruiter integrations
- Draw architecture diagram: React ↔ FastAPI ↔ (parsing + embedding service) ↔ PostgreSQL

• Day 2 — Repo & Environment Setup

- GitHub repo `go-gulf-experts`, FastAPI skeleton with `/health` endpoint, React skeleton with branded landing page
- Set up Python virtualenv, install spaCy (`en_core_web_sm`), sentence-transformers, pdfplumber
- `.env` config for DB URL, model paths

• Day 3 — Dataset Collection

- Pull a public resume dataset (e.g., Kaggle "Resume Dataset") and a job-postings dataset
- If possible, supplement with sample Gulf-region job postings (construction, oil & gas, finance, IT, hospitality — common Gulf hiring sectors) for realistic skill taxonomy and demo content

• Day 4 — Data Cleaning

- Clean resume text (strip formatting artifacts, normalize whitespace)
 - Build a skills taxonomy covering both general tech/soft skills and Gulf-market-relevant ones (e.g., certifications like NEBOSH, trade licenses, language skills — Arabic/English)
 - **Day 5 — EDA**
 - Explore resume length distribution, most common skills/roles
 - Sanity-check: do similar-role resumes cluster together with embeddings?
 - **Day 6 — Database Schema**
 - Tables: `users`, `resumes`, `job_descriptions`, `matches` (`resume_id`, `job_id`, `score`, `matched_skills`, `missing_skills`, `created_at`)
 - Set up Postgres in Docker, run migrations (Alembic), test CRUD from FastAPI
 - **Day 7 — Review & Buffer**
 - Confirm skills taxonomy and dataset are usable; finalize brand assets before UI work in Week 3
-

Week 2 — NLP / Model Development

Goal: A working parsing + matching pipeline that turns (resume, job description) into a score + skill breakdown.

- **Day 8 — Resume Parsing**
- Build parser: PDF/DOCX → plain text (pdfplumber/python-docx)
- Extract sections heuristically (experience, education, skills, certifications)
- **Day 9 — Skill Extraction (Baseline)**
- Baseline: keyword/phrase matching against the Go Gulf Experts skills taxonomy (`spaCy PhraseMatcher`)
- Test on sample resumes; note false positives/negatives
- **Day 10 — Skill Extraction (NER Upgrade)**
- Improve extraction using `spaCy` NER + custom rules
- Compare precision/recall against Day 9 baseline
- **Day 11 — Embedding-Based Matching**
- Embed resume text and job description text with Sentence-Transformers
- Compute cosine similarity as the core "match score"
- Sanity check against clearly good/bad pairs (e.g., a project engineer resume vs a hospitality JD should score low)
- **Day 12 — Score Calibration & Skill Gap Logic**
- Combine embedding similarity + skill-overlap ratio into a final weighted match score
- Build "missing skills" logic; tune weights based on error analysis
- **Day 13 — Pipeline Packaging**

- Wrap into one function: `match(resume_text, jd_text) -> {score, matched_skills, missing_skills}`
 - Unit tests: empty resume, non-English text, huge PDF, garbled OCR text
 - **Day 14 — Review & Buffer**
 - Document known limitations; freeze pipeline version (`matcher_v1`)
-

Week 3 — Backend, Frontend, Integration

Goal: Full Go Gulf Experts app working locally end-to-end.

- **Day 15 — Upload & Parsing Endpoints**
 - `POST /resumes` (file upload, parses + stores), `POST /job-descriptions` (text input, stores)
 - Validate file types/size limits; handle parse failures gracefully
 - **Day 16 — Matching Endpoint + Auth**
 - `POST /match` (`resume_id` + `job_id` → runs pipeline, stores + returns result)
 - Add JWT auth so users only see their own resumes/matches
 - **Day 17 — History & API Polish**
 - `GET /matches` (user's match history), `GET /matches/{id}` (detail view)
 - OpenAPI docs, API-level tests
 - **Day 18 — Frontend: Branded Landing + Upload Flow**
 - Go Gulf Experts landing page (hero, value prop, how-it-works)
 - Resume upload page (drag-and-drop), job description input page
 - **Day 19 — Frontend: Results Dashboard**
 - Match score visualization, matched vs missing skills as tags
 - "Boost your Gulf application" section listing missing skills clearly
 - **Day 20 — Frontend: Auth & History**
 - Login/signup pages, protected dashboard route
 - History page: past matches with score, date, job title
 - **Day 21 — Full Integration Pass**
 - End-to-end: signup → upload resume → paste JD → see score/skills → see it in history
 - Fix CORS, file upload edge cases, token expiry handling
-

Week 4 — Testing, Deployment, Launch

Goal: Deployed, tested, documented, demo-ready Go Gulf Experts platform.

- **Day 22 — Testing**

- Backend: unit + integration tests (parsing edge cases, auth, matching endpoint)
 - Frontend: component tests for upload flow and dashboard
 - Manual QA: corrupted PDF, scanned/image-only PDF, huge file, empty JD
 - **Day 23 — Performance & Security**
 - Cache sentence-transformer model load (load once at startup)
 - File-size limits, rate limiting on `/match`, sanitize uploaded filenames
 - Target latency check (e.g., <3s per match)
 - **Day 24 — Containerization**
 - Dockerfile for FastAPI (bundle/download model weights at build)
 - Dockerfile for React build; `docker-compose` for local full-stack + Postgres
 - **Day 25 — Deployment**
 - Deploy backend to Railway/Render (watch memory for embedding model size)
 - Deploy frontend to Vercel under the Go Gulf Experts domain/branding
 - Set production env vars/secrets
 - **Day 26 — CI/CD & Monitoring**
 - GitHub Actions: run tests on push, build Docker images
 - Add error logging (Sentry) and uptime check on `/health`
 - **Day 27 — Documentation & Demo Prep**
 - README: setup steps, architecture diagram, API reference, pipeline explanation and limitations
 - Prepare demo: 2-3 sample resumes + Gulf job postings showing clearly good/bad matches
 - **Day 28 — Final Review & Launch**
 - Fresh-eyes walkthrough as a new candidate would experience it
 - Fix last bugs, polish copy and empty/error states
 - Ship: share the live Go Gulf Experts link, write a short project writeup/post
-

Domain-specific watch-outs

- **PDF parsing is the #1 failure point.** Scanned/image resumes have no text layer — plan an OCR fallback (Tesseract) or clearly require text-based PDFs.
- **Skills taxonomy quality drives everything**, especially for Gulf-specific certifications and trade/language requirements — invest real time on Day 4/9.
- **Model memory footprint matters at deploy time.** Use a small sentence-transformer model (e.g., `all-MiniLM-L6-v2`) to avoid hitting memory limits on Day 25.
- **Branding consistency:** keep the Go Gulf Experts name/logo/color scheme consistent from the Day 1 landing page mockup through to the final deployed UI, so the Week 4 demo feels like a finished product, not a prototype.